# The Euclidean algorithm is not optimal among uniform processes

Jason Davies*

October 23, 2026

**Abstract**

Reusing a stored Euclidean prefix, we derive $\gcd(a, b)$ from $a, b$ with $O(\log a / \log\log a)$ applications of the remainder function alone, uniformly for $a > b \geq 1$; Euclid's algorithm needs order $\log a$ in the worst case. These derivations bound the intrinsic complexity of gcd and coprimeness by the same order, refuting Moschovakis's conjectured logarithmic lower bounds for uniform processes; by his best-uniform-process theorem, a single uniform process attains this bound. The Main Conjecture for algorithms remains open. We also prove an $\Omega(\log\log a)$ lower bound for almost all coprime pairs and give a family of finite nondeterministic gcd programs with arbitrarily small logarithmic cost coefficients, which need only a fixed number of registers and no recursion. The bounds and the refutations are formalised in Lean 4; the best-uniform-process theorem is cited.

**AI transparency statement.** This paper was produced entirely by artificial intelligence, including the identification of the main result, its formalisation in Lean, and the writing of the manuscript. The human author initiated the broader research and directed the AI systems, but did not write the text. Details are given at the end of the paper.

## 1 Introduction

Van den Dries and Moschovakis asked whether the Euclidean algorithm is optimal among its peers, the algorithms that compute the gcd from the remainder function [9, p. 390]. Moschovakis's Main Conjecture asserts that for every algorithm $\alpha$ from rem and the zero test $\mathrm{eq}_0$ computing $\gcd(x, y)$ there is an $r > 0$ such that $\alpha$ makes more than $r \log x$ remainder calls on infinitely many pairs $x > y \geq 1$ [6, p. 2]. We refute its formulations for uniform processes, the book's more general model (Section 5); for algorithms it remains open. Throughout, "the book" means Moschovakis's monograph [6], and logarithms are to base 2.

**Derivations.** A *derivation* from $a, b$ is a finite list that starts with $a, b$ and in which each further entry is $\mathrm{rem}(x, y)$ for earlier entries $x, y$ with $y > 0$. Each further entry is one *instruction*, that is, one remainder call. Entries already in the list are *available* and may be reused; no constants and no other operations are supplied. Let $G(a, b)$ be the least number of instructions in a derivation containing $\gcd(a, b)$, and let $D(a, b) = G(a, b)$ for coprime inputs. Thus $G(a, 0) = 0$ and $D(a, 1) = 0$. Arithmetic used in the proofs, such as subtraction, sorting or comparison, only selects instructions; it is never itself an instruction. Thus $G$ measures the length of a derivation, not the work of finding one.

*jason@jasondavies.com

**Theorem 1.** *There is a constant $C$ such that for all integers $a \geq 4$ and $0 \leq b \leq a$,*

$$G(a,b) \leq C \frac{\log a}{\log \log a}.$$

*In particular this bound holds for $D(a,b)$ when $\gcd(a,b) = 1$.*

**Idea of the proof.** For sufficiently large $n$, let $a > b$ be coprime with $a \leq N = 2^n$. Store a Euclidean prefix $r_0 = a, r_1 = b, \ldots, r_h$, where $h = \lceil \sqrt{n} \rceil$. If the prefix reaches a positive remainder at most $N^{3/4}$, take one further remainder and recurse on the resulting smaller pair. Otherwise its consecutive pairs stay distinct modulo every $m > N^{1/3}$, so the prefix occupies at least $\sqrt{h}$ residue classes modulo $m$ (Lemma 3). A close pair of residues lets three remainder calls produce a positive value below $m/(\sqrt{h} - 1)$ (Lemma 2). Reusing the same prefix, we repeatedly shrink the modulus from $r_h$ to at most $N^{1/3}$ in $O(n/\log n)$ calls. Two recursive gcd derivations on inputs at most $N^{1/3}$ finish. If $T(n)$ is the worst derivation cost for inputs at most $2^n$, the resulting recurrence (5) gives $T(n) = O(n/\log n)$. Tuning the prefix length sharpens the upper coefficient to $6 + o(1)$ (Corollary 4).

**Consequences and further results.** Uniform processes are the book's abstract model of computation from given primitives; every deterministic or nondeterministic recursive program induces one [6, §4A, Proposition 4A.1]. Section 5 converts the derivations into small certificates. This bounds the intrinsic complexity of gcd and coprimeness, refutes Conjecture 3.4 of [5] and version (4F-5) of the Main Conjecture, and limits what the homomorphism method can prove. The book's best-uniform-process theorem then gives a single process with the same bound on every input. The upper bound also settles an open problem of van den Dries and Moschovakis on the closure depth of the gcd (Corollary 5).

We also prove that $D(a,b) \geq (1 - o(1)) \log \log N$ for all but $o(N^2)$ coprime pairs $2 \leq b < a \leq N$ (Proposition 11). Finally, fixing the prefix length gives finite nondeterministic programs with arbitrarily small logarithmic coefficients, uniformly over all inputs (Theorem 12). Their program size grows as the requested coefficient decreases. They need no recursion: register programs with a fixed number of registers suffice (Theorem 13).

**Related work.** The known intrinsic lower bounds give $\Omega(\log \log a)$ on special families of coprime pairs [9, 10], [6, Theorem 6C.5], also for richer primitives such as addition and integer quotient, and for depth. Pratt's unpublished nondeterministic algorithm [6, Theorem 2E.2 and Problems x2E.17–x2E.19] matches this order on successive Fibonacci numbers [6, Corollary 6C.7]. For remainder-only derivations, our almost-all bound gives the same order on a set of full density among coprime pairs.

In a different model, Bshouty obtained the optimal bound $\Theta(\log N/\log \log N)$ for gcd on inputs in $[0, N]$ using a random access machine with linear operations, shifts, comparisons and indirect addressing [1, Theorems i and ii, p. 3]. His construction tabulates blocks of the binary gcd algorithm [8], [3, §4.5.2], computing table entries on demand and counting their construction [1, §4, Algorithm D and Lemma 5, pp. 12–14]. Here the saving comes from reusing a Euclidean prefix with remainder as the only instruction.

## 2 Normalisation

Write $g = \gcd(a,b) > 0$. For positive $y$,

$$\operatorname{rem}(gx, gy) = g \operatorname{rem}(x, y).$$

Every generated value is divisible by $g$. Consequently scaling a derivation, or dividing all its values by $g$, preserves its length and the validity of every instruction. Hence

$$G(a,b) = D(a/g, b/g). \tag{1}$$

Define

$$T(n) = \max_{0\le b\le a\le 2^n,\ a>0} G(a,b).$$

Euclid's remainders form a derivation containing $\gcd(a,b)$, so $G$ and $T$ are finite. By (1) it suffices to bound $D$; the cases $a=b$, $b=0$ and $b=1$ need no instructions.

## 3 Three-call contractions

**Lemma 2.** *If $m>0$ is available and a finite collection $S$ of available values occupies $t\ge 2$ residue classes modulo $m$, at most three instructions generate $w$ with*

$$0 < w < \frac{m}{t-1}.$$

*Proof.* Sort the distinct residues. Some adjacent pair $u<v$ has gap $\delta = v-u < m/(t-1)$, because the sum of the $t-1$ adjacent gaps is less than $m$. Each selected residue is $\mathrm{rem}(x,m)$ for some $x\in S$, so it costs one instruction. If $u=0$, use $v$. If $0<u\le\delta$, use $u$. Otherwise $0<\delta<u$ and $v=u+\delta$, so $\mathrm{rem}(v,u)=\delta$. This uses at most three instructions. □

**Lemma 3** (Euclidean-prefix lemma)**.** *Let $r_0=a>r_1=b>\cdots>r_h>0$ be a Euclidean prefix of a coprime pair, with $r_{i-1}=q_ir_i+r_{i+1}$ and $q_i\ge 1$ for $1\le i<h$. For every integer $m>2a/r_h$, the $h$ pairs*

$$(r_i \bmod m, r_{i+1} \bmod m), \qquad 0\le i<h,$$

*are distinct. The prefix therefore occupies $t$ residue classes with $t^2\ge h$.*

*Proof.* Suppose the pairs at $i<j$ coincide. Set

$$M = \prod_{\ell=i+1}^{j} \begin{pmatrix} q_\ell & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} A & B \\ C & E \end{pmatrix}, \qquad \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = M \begin{pmatrix} r_j \\ r_{j+1} \end{pmatrix}.$$

Right multiplication by $\left(\begin{smallmatrix} q & 1 \\ 1 & 0 \end{smallmatrix}\right)$ sends $(A,B,C,E)$ to $(qA+B,\ A,\ qC+E,\ C)$, starting from $(q,1,1,0)$ for a single factor. By induction, all entries are nonnegative, $A\ge B\ge E$, $A\ge C\ge 1$, $B\ge 1$, and $\det M = (-1)^{j-i}$. Also $Ar_j\le r_i\le a$ and $r_j>r_h$ (as $j<h$), whence $A<a/r_h$. Write

$$\Delta = \det(M-I) = 1+\det M - A - E.$$

For an odd number of factors, $\Delta = -A-E<0$. By the same recursion, a product of at least two factors has $A\ge 2$ and $E\ge 1$; thus for an even positive number of factors, $\Delta = 2-A-E<0$. In either case,

$$0<|\Delta|\le A+E\le 2A\le 2a/r_h<m.$$

The presumed congruence gives $(M-I)(r_j,r_{j+1})^{\mathsf{T}}\equiv 0 \pmod m$. Multiplication by the adjugate shows that $m$ divides both $\Delta r_j$ and $\Delta r_{j+1}$. Consecutive remainders are coprime, so Bézout's identity implies $m\mid\Delta$, a contradiction. Finally, $t$ individual residue classes give at most $t^2$ ordered pairs. □

Since Lemma 3 holds for every $m > 2a/r_h$, one stored prefix serves every later modulus, although these moduli are not known in advance. When $\sqrt{h} > 1$, Lemma 2 applied to the prefix gives three instructions that generate

$$0 < w < \frac{m}{\sqrt{h} - 1}. \tag{2}$$

We call these three instructions a *contraction* of $m$, and $w$ the new modulus.

For example, from $44, 17$ one instruction stores the prefix $44, 17, 10$. Modulo $m = 10 > 2 \cdot 44/10$ its residues are $4, 7, 0$, and the close pair $4 < 7$ gives the contraction

$$\mathrm{rem}(44, 10) = 4, \qquad \mathrm{rem}(17, 10) = 7, \qquad \mathrm{rem}(7, 4) = 3,$$

with new modulus $3 < 10/(3 - 1)$, as in Lemma 2. One more instruction, $\mathrm{rem}(10, 3) = 1$, finishes. Euclid's algorithm also takes five instructions here; the saving appears only for large inputs.

## 4 Construction and recurrence

Fix an integer $n > 16$ and put $N = 2^n$ and $h = \lceil \sqrt{n} \rceil > 4$. After normalisation, assume $a > b > 1$ are coprime and $a \le N$. We use $N^{3/4}$ as the *early threshold* and $N^{1/3}$ as the *target modulus*. Any thresholds $N^\alpha, N^\beta$ with $\alpha < 1$, $\alpha + \beta > 1$ and $2\beta < 1$ would do: these make the early recursive problem smaller, let the prefix lemma apply above the target, and make the two final problems together smaller. We take $\alpha = 3/4$ and $\beta = 1/3$.

**Phase (i): the prefix.** Starting from $r_1 = b$, generate the Euclidean remainders $r_2, r_3, \ldots$, one instruction each, until the most recent value is at most $N^{3/4}$ or $r_h$ is available. This costs at most $h$ instructions. Since the inputs are coprime, Euclid's algorithm reaches 1 before 0, so the stopping value is positive.

If the stopping value $c$ is at most $N^{3/4}$, generate the next remainder $s < c$ with one more instruction. The pair $(c, s)$ is coprime, with both entries at most $2^{\lceil 3n/4 \rceil}$. This gives the bound

$$h + 1 + T(\lceil 3n/4 \rceil). \tag{3}$$

**Phase (ii): contractions.** Otherwise $r_h > N^{3/4}$, so $2a/r_h < 2N^{1/4} < N^{1/3}$ (as $n > 12$), and Lemma 3 applies to every available $m > N^{1/3}$. Starting at $m = r_h$, repeatedly replace $m$ by the value $w$ of (2) until $0 < m \le N^{1/3}$. Set

$$\kappa = \sqrt{h} - 1 > 1, \qquad J = \left\lceil \frac{2n/3}{\log \kappa} \right\rceil.$$

There are at most $J$ contractions: if $J$ have occurred then $m < N/\kappa^J \le N^{1/3}$. Since $\log \kappa = \frac{1}{4} \log n + O(1)$, their cost is $3J = O(n/\log n)$. This is the source of the saving: the prefix costs only $O(\sqrt{n})$ instructions, and each contraction reduces $\log m$ by at least $\log \kappa = \frac{1}{4} \log n + O(1)$.

**Phase (iii): two recursive derivations.** The small $m$ need not be coprime to either input. Generate $u = \mathrm{rem}(a, m)$ and $v = \mathrm{rem}(b, m)$, then append a derivation of $d = \gcd(m, u)$ from $m, u$ and one of $e = \gcd(d, v)$ from $d, v$. Since $m \le 2^{\lceil n/3 \rceil}$ and $G$ is symmetric, each costs at most $T(\lceil n/3 \rceil)$. Finally

$$e = \gcd(\gcd(m, \mathrm{rem}(a, m)), \mathrm{rem}(b, m)) = \gcd(m, \gcd(a, b)) = 1.$$

The cost in this case is at most

$$h + 3J + 2 + 2T(\lceil n/3 \rceil). \tag{4}$$

**The recurrence.** The two case estimates (3) and (4) and normalisation imply

$$T(n) \le \max\{T(\lceil 3n/4 \rceil) + O(\sqrt{n}),\ 2T(\lceil n/3 \rceil) + O(n/\log n)\}. \tag{5}$$

All implied constants are independent of the inputs.

*Proof of Theorem 1.* For $n \ge 2$ let $f(n) = n/\log n$. The ratios

$$\frac{f(\lceil 3n/4 \rceil)}{f(n)} \longrightarrow \frac{3}{4}, \qquad \frac{2f(\lceil n/3 \rceil)}{f(n)} \longrightarrow \frac{2}{3}$$

show that, beyond a fixed threshold, $f(\lceil 3n/4 \rceil) \le \frac{7}{8} f(n)$ and $2f(\lceil n/3 \rceil) \le \frac{5}{6} f(n)$. Both overheads in (5) are at most $c_0 f(n)$ for a fixed $c_0$. Choose the induction threshold above 16, where the recurrence holds, and large enough that both recursive indices are at least 2 and strictly less than $n$. Choose $C \ge 8c_0$, increased to cover the finitely many values below the threshold. Strong induction then gives $T(n) \le Cf(n)$: the two bounds are $(7C/8 + c_0)f(n)$ and $(5C/6 + c_0)f(n)$, both at most $Cf(n)$. Finally take $n = \lceil \log a \rceil$. For $a \ge 4$, $n \le 2 \log a$ and $\log n \ge \log \log a \ge 1$, so $G(a, b) \le T(n) \le 2C \log a / \log \log a$. □

### 4.1 The leading coefficient

The same construction allows a more precise bound without changing either arithmetic lemma.

**Corollary 4.** *For every $\varepsilon > 0$, there is an $a_0$ such that for every $a \ge a_0$ and $0 \le b \le a$,*

$$G(a, b) \le (6 + \varepsilon) \frac{\log a}{\log \log a}.$$

*Equivalently, the uniform upper bound has coefficient $6 + o(1)$.*

*Proof.* Fix $0 < c < 1/2$ and $0 < \beta < 1/2$, and put $\alpha = 1 - \beta/2$. For large $n$ put

$$p = \lfloor c \log n \rfloor, \qquad k = 2^p, \qquad h = k^2 + 1, \qquad J = \lceil n/p \rceil.$$

Use $N^\alpha$ as the early threshold and $N^\beta$ as the target modulus, where $N = 2^n$. In the late case $r_h > N^\alpha$, so $2a/r_h < 2N^{1-\alpha} = 2N^{\beta/2} < N^\beta$ for sufficiently large $n$, and the prefix lemma applies to every modulus above $N^\beta$. Since $k^2 < h$, the prefix occupies at least $k+1$ residue classes, so each contraction reduces the modulus by a factor greater than $k$. Moreover $k^J \ge N$, so $J$ contractions suffice. Writing $f(n) = n/\log n$, we have

$$h = O(n^{2c}) = o(f(n)), \qquad 3J = \left( \frac{3}{c} + o(1) \right) f(n).$$

The two branches therefore give

$$T(n) \le \max \left\{ T(\lceil \alpha n \rceil) + o(f(n)), \quad 2T(\lceil \beta n \rceil) + \left( \frac{3}{c} + o(1) \right) f(n) \right\}.$$

Theorem 1 implies that $\Lambda = \limsup_{n \to \infty} T(n)/f(n)$ is finite and nonnegative. For each fixed $q > 0$, the ratio $f(\lceil qn \rceil)/f(n)$ tends to $q$. Taking upper limits in the recurrence yields

$$\Lambda \le \max\{\alpha\Lambda,\ 2\beta\Lambda + 3/c\}.$$

Since $\alpha < 1$ and $\Lambda \ge 0$, this implies $(1 - 2\beta)\Lambda \le 3/c$. First let $\beta \downarrow 0$ and then $c \uparrow 1/2$ to obtain $\Lambda \le 6$. Finally, $n = \lceil \log a \rceil$ satisfies $f(n) \sim \log a / \log \log a$, proving the assertion. All upper limits concern the worst-case function $T$, so the resulting threshold is independent of $b$. □

We do not claim that 6 is optimal.

### 4.2 Closure depth

Van den Dries and Moschovakis [10] measure the parallel depth of the gcd. Put $S_0 = \{0, 1, a, b\}$, let $S_{k+1}$ adjoin to $S_k$ all values $x+y$, $x-y$, $\mathrm{iq}(x,y)$ (integer quotient) and $\mathrm{rem}(x,y)$ with $x, y \in S_k$, and let $g(a,b)$ be the least $k$ with $\gcd(a,b) \in S_k$ [10, p. 2:6]. They ask whether there are positive integers $\alpha_n \to \infty$ and $\beta_n$ and a $C > 0$ with $g(\alpha_n, \beta_n) > C \log \alpha_n$ for all $n$ [10, Open Problem, p. 2:11].

**Corollary 5.** *For integers $\alpha \geq 4$ and $\beta \geq 1$,*

$$g(\alpha, \beta) \leq 1 + G(\alpha, \mathrm{rem}(\beta, \alpha)) = O\left(\frac{\log \alpha}{\log \log \alpha}\right).$$

*Hence no such sequences exist.*

*Proof.* Put $r = \mathrm{rem}(\beta, \alpha)$, so $r \in S_1$, $0 \leq r < \alpha$ and $\gcd(\alpha, r) = \gcd(\alpha, \beta)$. In a derivation from $\alpha, r$, the value of the $k$th instruction lies in $S_{k+1}$, by induction on $k$: its operands are $\alpha$, $r$ or earlier values, all in $S_k$. The source's remainder agrees with ours at positive divisors. Theorem 1 bounds $G(\alpha, r)$. □

## 5 Certificates and uniform processes

**Structures and certificates.** Write $\mathrm{tt}, \mathrm{ff}$ for the truth values and $\mathrm{eq}_c$ for the test $x = c$. We use the book's partial remainder operation, defined only when its second argument is nonzero, in the structures

$$\mathcal{N}_0 = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0), \qquad \mathcal{N}_\varepsilon = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1).$$

The latter is the book's Euclidean structure $\mathbf{N}_\varepsilon$ [6, (1C-10), p. 24]; our $\mathcal{N}_0$ should not be confused with the book's $\mathbf{N}_0$, which has no remainder operation [6, p. 171]. Coprimeness is the relation $x \perp\!\!\!\perp y \iff x, y \geq 1 \wedge \gcd(x, y) = 1$ [6, §1A].

A derivation records finitely many remainder equations. These form a partial structure generated by the inputs. A *certificate* for $f$ at $\vec{x}$ in a structure $\mathcal{A}$ is such a finite generated partial substructure, possibly with test facts, on which every homomorphism into $\mathcal{A}$ respects $f$ at $\vec{x}$. In other words, any assignment of values preserving the recorded operations and tests must preserve the answer. The *intrinsic remainder-call complexity* $\mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, f, \vec{x})$ is the least number of remainder facts in a certificate [6, §§1D, 4C–4E].

The Lean development represents a certificate by its facts. The following lemma justifies this, and supplies the derivations used below.

**Lemma 6** (Fact representation)**.** *Let $F$ be a finite set of true remainder and test facts in which every number is generated from $a, b$ by the remainder facts of $F$. Then $F$ is the diagram of a finite partial substructure generated by $a, b$ (of $\mathcal{N}_\varepsilon$, or of $\mathcal{N}_0$ if $F$ has no $\mathrm{eq}_1$ facts), and every such substructure arises in this way. A map $\pi : \mathbb{N} \to \mathbb{N}$ preserves the facts of $F$ exactly when its restriction to the universe is a homomorphism, and every homomorphism extends to such a map. Every element of the universe is the last value of a derivation from $a, b$ with at most as many instructions as $F$ has remainder facts.*

*Proof.* Take the universe $V$ to consist of $a, b$ and the numbers occurring in $F$, and interpret each primitive only at the recorded arguments, with the recorded value. Truth of the facts makes these partial operations and tests restrictions of the primitives of $\mathbb{N}$, and the hypothesis says that $V$ is generated by $a, b$. Conversely, the diagram of a finite generated partial substructure is such a set

$F$. A homomorphism need only respect the recorded operations and tests on $V$, and values outside $V$ are irrelevant; this gives the second assertion. For the last, start from $a, b$ and repeatedly apply a remainder fact of $F$ whose operands are available and whose value is new. While some element of $V$ is unavailable such a fact exists: take the first unavailable value in a generation of that element. Each fact is applied at most once. □

**Proposition 7.** *For coprime $a, b \geq 1$, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) = D(a, b)$. For $(a, b) \neq (0, 0)$,*

$$G(a, b) \leq \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b) \leq G(a, b) + 2,$$

*and the same bounds hold for $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$.*

*Proof.* A derivation of 1, together with $\mathrm{eq}_1(1) = \mathrm{tt}$, $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$, certifies $\perp\!\!\!\perp$: under any homomorphism the input images are positive, and each of their common divisors divides every transported generated value, hence divides the image 1. Conversely, if a certificate has no positive $\mathrm{eq}_1$ fact, then $x \mapsto 2x$ preserves its remainder facts, zero tests and negative one-tests, but not coprimeness. So every certificate contains 1, and Lemma 6 gives a derivation of 1 with at most as many instructions as the certificate has remainder facts. This proves the first assertion.

For gcd as a function, the certificate must reach its output $g$, giving a lower bound of $G(a, b)$. For an upper bound, append

$$\mathrm{rem}(a, g) = z, \qquad \mathrm{rem}(b, g) = z, \qquad \mathrm{eq}_0(z) = \mathrm{tt},$$

where the generated $z$ is zero. Every homomorphism $\pi$ has $\pi(z) = 0$ and $\pi(g) > 0$ (by definedness of either appended remainder). Thus $\pi(g)$ divides both input images. Conversely every common divisor of those images divides $\pi(g)$ by induction over the derivation. This proves the bounds for $\mathcal{N}_0$. The lower-bound argument applies to every gcd certificate, $\mathrm{eq}_1$ facts included, and allowing $\mathrm{eq}_1$ facts can only lower the count, so the upper bound transfers to $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$. □

*Remark* 8. At $(0, 0)$ the output is an input, and the substructure with universe $\{0\}$ and no facts is a certificate. For gcd restricted to positive inputs, appending $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$ keeps input images in the domain, at no remainder cost. For a positive noncoprime input, adding $\mathrm{eq}_1(g) = \mathrm{ff}$ to the gcd certificate certifies noncoprimeness, so the sublogarithmic remainder-call bound holds for both truth values of coprimeness as well as for the gcd. The overhead is a fixed additive constant and does not change the leading coefficient, so Corollary 4 holds for all three complexities of Corollary 10 below on positive inputs.

**A single uniform process.** Proposition 7, Remark 8, and Theorem 1 bound each intrinsic complexity by $O(\log a / \log\log a)$, uniformly for $a \geq b \geq 1$. The book's Theorem 4F.1 states that, whenever every input has a certificate, one uniform process attains the intrinsic complexity at every input [6, pp. 149–150]. Hence:

**Corollary 9.** *For each of gcd and coprimeness, there is a single uniform process of $\mathcal{N}_\varepsilon$ computing it on $\mathbb{N}^2$ with $O(\log a / \log\log a)$ remainder calls, uniformly for $a \geq b \geq 1$. The gcd process can also be taken over $\mathcal{N}_0$.*

*Proof.* The certificate bounds give the cost assertion by Theorem 4F.1. Every input has a certificate: symmetry covers reversed inputs, and a zero-test fact suffices when either input is zero. Thus the same process is defined on all of $\mathbb{N}^2$. □

The corollary rules out the possibility raised after Corollary 6C.7 of the book that the Main Conjecture holds for all uniform processes of $\mathcal{N}_\varepsilon$ [6, p. 188].

**The conjectures refuted.** Moschovakis's 2012 notes state:

> **Conjecture 3.4** [5, p. 19]. There is a rational number $r > 0$ such that for infinitely many $a \geq b \geq 1$, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \geq r \log a$.

The book formulates one precise version of the Main Conjecture [6, p. 151]: there are a set $\mathcal{D} \subseteq \mathbb{N}^2$ and a $K > 0$ such that

$$(x, y) \in \mathcal{D} \implies \mathrm{calls}_{\mathrm{rem}}(\mathrm{Eucl}, x, y) \leq L(x, y) \leq K \, \mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y), \tag{4F-5}$$

where $L(x, y) = 2 \log \max(x, y)$ for $(x, y) \neq (0, 0)$, $L(0, 0) = 0$, and $\mathrm{calls}_{\mathrm{rem}}(\mathrm{Eucl}, x, y)$ counts the remainder calls of the Euclidean algorithm Eucl (the book writes $\varepsilon$) on $(x, y)$. The book states (4F-5) for the Euclidean algorithm in $\mathbf{N}_\varepsilon$, so its $\mathcal{A}$ is $\mathcal{N}_\varepsilon$; we treat both $\mathcal{A} = \mathcal{N}_\varepsilon$ and $\mathcal{A} = \mathcal{N}_0$. The book adds that $\mathcal{D}$ then satisfies its conditions (4F-3) and (4F-4); the latter makes the intrinsic complexity unbounded on $\mathcal{D}$, so $\mathcal{D}$ is infinite (for finite $\mathcal{D}$ the statement is trivial). We refute the weaker statement obtained by dropping the first inequality and requiring only that $\mathcal{D}$ be infinite.

**Corollary 10.** *The intrinsic complexities*

$$\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$$

*are $O(\log a / \log \log a)$, uniformly for $a \geq b \geq 1$. Consequently Conjecture 3.4 of [5] is false, and so is the version (4F-5) of the Main Conjecture with $\mathcal{A} = \mathcal{N}_0$ or $\mathcal{A} = \mathcal{N}_\varepsilon$: for every $r > 0$ only finitely many pairs $a \geq b \geq 1$ satisfy $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \geq r \log a$, and for every $K > 0$ only finitely many pairs $(x, y) \in \mathbb{N}^2$ satisfy $L(x, y) \leq K \, \mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y)$.*

The corollary follows from Theorem 1 and Proposition 7; for noncoprime pairs, the bound on the coprimeness complexity comes from Remark 8. For (4F-5), pairs with $x < y$ or a zero entry are handled as in the proof of Corollary 9; the latter need no remainder facts.

**The limit of the homomorphism method.** The Homomorphism Test proves lower bounds on intrinsic complexity by showing that every generated diagram below the proposed cost admits a homomorphism that fails to preserve the answer [6, Lemma 4E.3]. Such bounds apply to every uniform process computing the function, hence to every recursive program [6, Proposition 4E.1, Corollary 4E.2]. By Corollary 10, no lower bound on the intrinsic complexities listed there can grow faster than $\log a / \log \log a$, so a logarithmic lower bound for algorithms must use restrictions beyond uniformity, such as finite program syntax or determinism.

## 6 An almost-all lower bound

The difficult pairs of [6, Theorem 6C.5] give $\Omega(\log \log a)$ on a sparse set. An elementary Diophantine argument gives the same order for almost all coprime pairs. Write each generated value as $ua + wb$. On the $B$-regular pairs defined below, small coefficients cannot give a small positive value, and each remainder call at most squares the coefficient bound, up to a factor $2B$. Reaching 1 therefore takes about $\log \log a$ calls, except on a sparse set of pairs.

**Proposition 11.** *For integers $d \geq 1$ and $N \geq 3$, at most $40(2 + \ln N) N^{2 - 2^{-d}}$ coprime pairs $2 \leq b < a \leq N$ satisfy $D(a, b) \leq d$. Hence, for each fixed $\varepsilon > 0$, all but $o(N^2)$ of these pairs satisfy $D(a, b) > (1 - \varepsilon) \log \log N$.*

*Proof.* For $B \geq 1$ call $(a, b)$ *$B$-regular* if $|qb - pa| \geq a/(Bq)$ whenever $q \in \mathbb{N}$, $q \geq 1$, $p \in \mathbb{Z}$ and $qb \neq pa$. We first bound the cost on regular pairs, then count the irregular ones.

Every generated value has an integer representation $v = ua + wb$. On a regular pair, $v > 0$ implies $v \geq a/(B(|u| + |w|))$: for $w \neq 0$ use regularity with $q = |w|$, and for $w = 0$ note $v \geq a$. Let $H_t$ bound the coefficient norm $|u| + |w|$ in one such representation of each value available after $t$ instructions, starting with $H_0 = 1$. In an instruction $r = \mathrm{rem}(x, y)$, write $x = sy + r$. Since generated values are at most $a$, the quotient satisfies $s \leq a/y \leq BH_t$. Subtracting $s$ times the representation of $y$ from that of $x$ gives

$$H_{t+1} \leq H_t + BH_t^2 \leq 2BH_t^2, \qquad H_t \leq (2B)^{2^t - 1}.$$

Producing 1 within $d$ instructions therefore forces

$$1 \geq \frac{a}{BH_d}, \qquad a \leq \frac{(2B)^{2^d}}{2}.$$

To count irregular pairs below $N$, choose a witness $0 < |qb - pa| < a/(Bq)$. Dividing $p, q$ by their gcd preserves this inequality, so assume $\gcd(p, q) = 1$. The nonzero integer $r = qb - pa$ then gives $q < a/B \leq N/B$ and $0 \leq p \leq q$. For each $q$, there are at most $2q$ choices of $p$ and $2N/(Bq)$ choices of the signed integer $r$. Given $p, q, r$, the equation $pa \equiv -r \pmod{q}$ determines one residue class for $a$, leaving at most $N/q + 1$ choices; $b$ is then determined. With $Q = \lfloor N/B \rfloor$, the number of irregular pairs is at most

$$\begin{aligned}\sum_{q=1}^{Q} (2q) \frac{2N}{Bq} \left( \frac{N}{q} + 1 \right) &= \frac{4N^2}{B} \sum_{q=1}^{Q} \frac{1}{q} + \frac{4NQ}{B} \\ &\leq \frac{4N^2}{B} (2 + \ln N),\end{aligned}$$

using $Q \leq N$, $B \geq 1$, and the harmonic-sum bound; an empty sum is zero.

Now split into dyadic ranges $X/2 < a \leq X$, and take $B = X^{2^{-d}}/2$. If $B \geq 1$, a regular pair with $D \leq d$ would have $a \leq X/2$, so all such pairs in the range are irregular. Their number is at most $8(2 + \ln X) X^{2 - 2^{-d}}$. When $B < 1$, the same bound follows from the trivial bound $X^2$, since $X^{2^{-d}} < 2$. Put $s = 2 - 2^{-d} \geq 3/2$. Summing over the nonempty ranges with $X = N/2^j \geq 3$, and bounding their logarithmic factors by $2 + \ln N$, gives

$$8(2 + \ln N) N^s \sum_{j \geq 0} 2^{-js} = \frac{8}{1 - 2^{-s}} (2 + \ln N) N^s \leq 40(2 + \ln N) N^s.$$

For the second assertion, take $d = \lfloor \log \ln N - 2 \log \ln \ln N \rfloor$ for sufficiently large $N$. Then $N^{-2^{-d}} \leq \exp(-(\ln \ln N)^2)$, so the exceptional set has size $o(N^2)$, while $d = (1 - o(1)) \log \log N$, so eventually $d \geq (1 - \varepsilon) \log \log N$. □

A positive proportion (asymptotically $6/\pi^2$) of these pairs are coprime, so the exceptional set also has density zero among coprime pairs. The saving $2^{-d}$ in the exponent of $N^{2 - 2^{-d}}$ shrinks as $d$ grows, so this argument does not reach the upper-bound scale $\log N / \log \log N$.

## 7 Fixed programs with arbitrarily small coefficients

For a nondeterministic program $P$, let $c_P(a, b)$ be the minimum number of remainder calls among its convergent computations. We require at least one such computation on each input and require every convergent computation to return the gcd. These are the conventions of [6, (2E-2), (2E-5)].

**Theorem 12.** *For each integer $p \geq 1$, there is a finite nondeterministic recursive program $P_p$ over $\mathcal{N}_0$ computing the gcd such that*

$$\limsup_{a\to\infty} \max_{1\leq b\leq a} \frac{c_{P_p}(a,b)}{\log a} \leq \frac{3}{p}.$$

*Its generator, defined below, has $W = 4^p + 5$ individual parameters, and no internal function has larger arity. Consequently, for every $\varepsilon > 0$ there are a fixed program $P$ and a constant $B \geq 0$ such that*

$$\forall a \geq b \geq 1, \qquad c_P(a,b) \leq \varepsilon \log a + B.$$

The program depends on $\varepsilon$ but not on the inputs. The Main Conjecture lets its coefficient $r$ depend on the program, so the family is consistent with it; no single sublogarithmic program is obtained.

**The program.** Fix $W \geq 3$ and write $\vec{x} = (x_0, \ldots, x_{W-1})$ and $\vec{x}[i \leftarrow E]$ for the tuple with its $i$th expression replaced by $E$. The generator can return a parameter, replace one by a remainder, run a recursive subcomputation while retaining the caller's other values, or copy parameters into new positions. These four actions are expressed by the finite alternatives

$$\begin{aligned}
\mathrm{choose}(\vec{x}) &= x_i && (0 \leq i < W), && \text{(C)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{choose}(\vec{x}), && && \text{(G1)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x}))]) && (0 \leq i < W), && \text{(G2)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{gen}(\vec{x})]) && (0 \leq i < W), && \text{(G3)}\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\mathrm{choose}(\vec{x}), \ldots, \mathrm{choose}(\vec{x})). && && \text{(G4)}
\end{aligned}$$

There are $W$ independent selector calls in (G4). Tuple notation and indices are syntax abbreviations: after fixing $W$, each line expands into ordinary first-order terms with fixed arities. No tuple, index, numeric constant, or update operation is supplied as runtime data or a primitive. The nested generator in (G3) is evaluated once; its caller's other parameters remain available when it returns. The recursion stack can grow, so $W$ is an arity bound, not a total storage bound.

Add $\mathrm{loop}(z) = \mathrm{loop}(z)$ and define $\mathrm{accept}(a,b,g)$ by nested zero-test conditionals: if $g = 0$, loop; otherwise test $\mathrm{rem}(a,g) = 0$ and then $\mathrm{rem}(b,g) = 0$, returning $g$ if both tests succeed and looping otherwise. The head is

$$P_W(a,b) = \text{if } \mathrm{eq}_0(b) \text{ then } a \text{ else } \mathrm{accept}(a,b,\mathrm{gen}(a,b,a,\ldots,a)).$$

The four internal symbols have arities $W, W, 3, 1$ (for choose, gen, accept, loop). Every common divisor of the generator's parameters divides its output, by induction on a finite evaluation. Thus a candidate is divisible by $\gcd(a,b)$; successful verification proves the reverse divisibility. Every convergent computation therefore returns the gcd. There is always a convergent computation following Euclid, using parameter selection and copying to retain the current pair. For $b > 0$, final verification costs exactly two remainder calls.

This syntax is a finite recursive program in the sense of [6, §§2A, 2E]; Appendix A records the Lean check that its evaluation trees correspond, with equal remainder cost, to convergent computations of a specialisation of the book's recursive machine.

*Proof of Theorem 12.* We follow the construction of Section 4: store the prefix, contract, then recurse twice through nested generator calls. Take $P_p = P_W$ with $W = 4^p + 5$; that is, set $k = 2^p$,

$h = k^2+1$ (the prefix length, as in Corollary 4 but now with $p$ fixed), and $W = h+4$. Store $r_0, \dots, r_h$ in $h+1$ parameters, reserving three parameters for the modulus and two scratch residues. Lemmas 2 and 3 give a contraction by a factor greater than $k$ in at most three remainder calls, preserving the prefix. The program can choose precisely the operands of each contraction.

The *generator cost* of $(a, b)$ is the minimum number of remainder calls in an evaluation of gen that returns $\gcd(a, b)$ from $(a, b, a, \dots, a)$. Let $T_W(n)$ be its maximum over $a, b \le 2^n$. Euclid gives $T_W(n) = O(n)$. To *route* to $x, y$ means to apply (G4) to pass to the tuple $(x, y, x, \dots, x)$, at no remainder cost. Copying parameters gives symmetry. Multiplying every value by $g = \gcd(a, b)$ turns a run on $(a/g, b/g)$ that returns 1 into a run on $(a, b)$ that returns $g$, with the same remainder count. It therefore suffices to analyse coprime inputs.

Fix $0 < \beta < 1/2$ and put $\alpha = 1 - \beta/2$. Use the early threshold $N^\alpha$ and the target modulus $N^\beta$, where $N = 2^n$. If no prefix remainder is at most $N^\alpha$, then $r_h > N^\alpha$ and $2a/r_h < 2N^{1-\alpha} < N^\beta$ for all sufficiently large $n$. There are at least $k+1$ distinct prefix residues modulo every modulus used before stopping, since $h > k^2$. At most $\lceil n/p \rceil$ contractions produce a positive $m \le N^\beta$.

In the early case, route directly to the smaller consecutive Euclidean pair. Otherwise form $u = a \bmod m$ and $v = b \bmod m$ in the scratch positions. A nested generator call, with its parameters routed to copies of $m, u$, returns $d = \gcd(m, u)$ while its caller retains $v$. Replace the modulus parameter by $d$ and route to copies of $d, v$ for the second gcd computation. The result is $\gcd(m, a, b) = 1$. All these routes and nested calls are available in the same fixed program. Thus, eventually in $n$,

$$T_W(n) \le \max\left\{T_W(\lceil \alpha n \rceil) + h + 1, \quad 2T_W(\lceil \beta n \rceil) + h + 3\lceil n/p \rceil + 2\right\}.$$

Set $\Lambda_W = \limsup_{n\to\infty} T_W(n)/n$, finite and nonnegative by Euclid. Since $h$ is fixed, division by $n$ and passage to upper limits gives

$$\Lambda_W \le \max\{\alpha\Lambda_W,\ 2\beta\Lambda_W + 3/p\}.$$

As $\alpha < 1$ and $\Lambda_W \ge 0$, this implies $(1-2\beta)\Lambda_W \le 3/p$. The program is independent of $\beta$, so letting $\beta \downarrow 0$ gives $\Lambda_W \le 3/p$. Take $n = \lceil \log a \rceil$ and add the two verification calls. Finally, choose $p > 3/\varepsilon$ and absorb the finitely many smaller inputs in $B$. □

**Bounded memory.** The nested calls in the proof are not needed. A *register program* keeps its values in a fixed number of registers and executes assignments $x_l := E$, branches on a test $E$, and a halt returning $E$, where each $E$ is a term in the registers; the nondeterministic version allows several commands at a line [6, §§2E–2F]. It is an iterator, so it uses no recursion stack, and in pointed structures register programs compute exactly the tail-recursive functions [6, Proposition 2F.1]. Its cost $c_P$ counts remainder evaluations as above.

**Theorem 13.** *For each integer $p \ge 1$, there is a finite nondeterministic register program over $\mathcal{N}_0$ with $4^p + 7$ registers that computes the gcd on inputs $a \ge b \ge 1$ and satisfies the bound of Theorem 12:*

$$\limsup_{a\to\infty} \max_{1\le b\le a} \frac{c_P(a, b)}{\log a} \le \frac{3}{p}.$$

*Proof.* Put $k = 2^p$, $h = k^2 + 1$ and $W = h + 6$, and start the registers $x_0, \dots, x_{W-1}$ at $(a, b, a, b, a, \dots, a)$. At its main line the program may perform any assignment $x_l := \mathrm{rem}(x_i, x_j)$ with $i, j, l \ge 2$, or pass to verification with a candidate $x_i$, $i \ge 2$. Verification halts with $x_i$ if $x_i \ne 0$, $\mathrm{rem}(x_0, x_i) = 0$ and $\mathrm{rem}(x_1, x_i) = 0$, and loops otherwise. The inputs $x_0, x_1$ are never overwritten. The proof below describes computations that move values between registers; no copying command

is needed, since a computation can leave each value in place and write each new remainder to a register whose value is no longer used, recording the roles of the registers in its choice of commands. Every register value is divisible by $\gcd(a, b)$, so every convergent computation returns the gcd, and Euclid's algorithm gives one.

As before, scaling reduces the analysis to coprime pairs. Let $S(n)$ be the largest, over coprime $1 \le b \le a \le 2^n$, of the least number of remainder calls that, starting from any state with $a, b$ in two working registers, place 1 in a working register. Fix $0 < \beta < 1/2$, put $\alpha = 1 - \beta/2$ and $N = 2^n$. Compute the Euclidean remainders $r_0 = a, r_1 = b, r_2, \ldots$ in working registers, stopping at $r_h$ or at the first $r_j \le N^\alpha$. In the latter case one further remainder gives the coprime pair $r_j, r_{j+1}$, bounded by $N^\alpha$, at total cost at most $h$; continue with that pair. Otherwise $r_h > N^\alpha$, and as in the proof of Theorem 12, at most $\lceil n/p \rceil$ contractions, using a modulus register and two scratch registers, produce a positive $m \le N^\beta$. Then put $u = a \bmod m$ and $v = b \bmod m$, and run Euclid's algorithm in place on $m, u$ and then on the result $d$ and $v$. Each run takes at most $2 \log m + 3$ calls, and the result is $\gcd(d, v) = \gcd(m, a, b) = 1$. Thus, eventually in $n$,

$$S(n) \le \max\left\{S(\lceil \alpha n \rceil) + h, \quad h + 3\lceil n/p \rceil + 4\beta n + 7\right\}.$$

Euclid gives $S(n) = O(n)$. Passing to upper limits, $\Lambda = \limsup_{n\to\infty} S(n)/n$ satisfies $\Lambda \le \max\{\alpha\Lambda,\ 3/p + 4\beta\}$, and since $\alpha < 1$, $\Lambda \le 3/p + 4\beta$. The program does not depend on $\beta$, so $\Lambda \le 3/p$. Adding the two verification calls proves the bound. □

Here the completion uses Euclid's algorithm, whose cost $O(\beta n)$ is negligible as $\beta \to 0$; the recursive completion of Section 4 is needed only when the prefix length grows.

## 8 Discussion and open problems

**The gap.** The intrinsic complexity of coprimeness in $\mathcal{N}_\varepsilon$ is at least $\Omega(\log \log a)$ on almost all coprime pairs and at most $O(\log a / \log \log a)$ on all positive pairs. Closing this gap is open; unlike Bshouty's bound in his model, our upper bound is not known to be optimal.

**From a growing prefix to one program.** The remaining obstacle in turning the construction into a sublogarithmic program is reusable access to a prefix whose length grows with the input. Selecting a close pair of residues costs nothing in a derivation, whereas computing all prefix residues would take about $h$ calls per contraction. The programs of Section 7 select operands nondeterministically from a fixed number of registers. Can one fixed first-order recursive program retain and repeatedly access a growing prefix at the same remainder cost? Proposition 2E.1 of the book [6, §2E] converts each certificate into a computation of some program, not of one program for all certificates.

**Memory, not recursion.** By Theorem 13, $W$ registers give coefficient $3/p$ with $W = 4^p + 7$, roughly $6/\log W$. So no bound $c \log a$ with $c$ independent of the number of registers holds, even for nondeterministic register programs. Does every nondeterministic register program with $W$ registers make at least $c \log a / \log W$ remainder calls on infinitely many inputs, for an absolute $c > 0$? Such a bound cannot rest on how quickly values shrink, since $\mathrm{rem}(m + 1, m) = 1$. Pratt's algorithm is itself a nondeterministic register program [6, Theorem 2E.2]; compare also the book's question whether recursion reduces to tail recursion in the pointed Euclidean structure [6, Problem x2G.6].

**Stronger lower-bound methods.** The limit established in Section 5 leaves room for arguments combining homomorphisms with additional restrictions on computations. In particular, the conjecture of a logarithmic lower bound for deterministic uniform processes remains open [5, Conjecture 3.9, p. 23], [6, §4H and p. 188].

## 9 Computational checks

The Python planner `remainder_only_certificate.py`, in the repository https://github.com/jasondavies/euclid-uniform-processes, chooses instructions offline with unrestricted arithmetic, using the parameters of Section 4; below a base size (256 bits by default) it uses Euclid's algorithm. An independent replay then checks operand availability, nonzero divisors, every remainder equation and the final gcd.

The script `scripts/benchmark_certificates.py` uses seed 20261016 to sample 25 coprime pairs at each size: $a$ is uniform among integers of that bit length, and $b$ is drawn uniformly from $\{2, \ldots, a-1\}$ until $\gcd(a, b) = 1$. It also runs the successive Fibonacci pair $(F_k, F_{k-1})$ with $F_k$ the first Fibonacci number of that bit length ($k = 738$, 2951, 11801). The table gives remainder calls; the Euclidean count stops when 1 first appears.

| | Random pairs: min / median / max | | Fibonacci pair | |
|---|---|---|---|---|
| Input bits | Construction | Euclid | Construction | Euclid |
| 512 | 189 / 214 / 226 | 266 / 295 / 337 | 38 | 735 |
| 2048 | 617 / 647 / 669 | 1136 / 1192 / 1260 | 60 | 2948 |
| 8192 | 2007 / 2037 / 2059 | 4664 / 4774 / 4890 | 110 | 11798 |

All 78 derivations and the example of Section 3 pass replay, and `validation/benchmark_certificates.json` records the individual counts. On the Fibonacci pairs, Euclid's worst case, the bits removed by successive contractions roughly double (16, 31, 64, 125 and 252 at 512 bits), so few contractions are needed. These samples illustrate the construction; the asymptotic claim rests on the proof. The accompanying repository also checks scaled and noncoprime inputs, rejects invalid traces, and exhaustively tests the prefix lemma over a finite range. For Section 7, a separate compiler and interpreter replay successful choices in finite first-order programs, evaluating arguments right to left as the book's machine does. The `README.md` records commands, test ranges, seeds, and results. The Python interpreter is not formally verified; unrestricted arithmetic used to plan choices is outside the programs.

## A Formal verification

The development uses Lean 4.35.0 [2] with mathlib [4] at its v4.35.0 tag; the pinned mathlib commit is `56a005855ea17e0b0958f73ec801ae140b0a1679`. The proved library contains no proof placeholders or added axioms; the main-result audit reports only `propext`, `Classical.choice`, and `Quot.sound`. Table 1 gives the main declarations; the repository README gives the complete map and reproduction commands. The general theory of uniform processes, including Theorem 4F.1 used in Corollary 9, and the historical lower bounds are cited, not formalised. The certificate bounds and conjecture refutations are formalised independently of that theory, with certificates represented by their facts as in Lemma 6; that lemma and Corollary 5 are proved here, not formalised.

The development is also packaged for the Palomar registry: `Challenge.lean` imports only mathlib and states the main theorems with every definition visible and deliberate `sorry` placeholders. `Solution.lean` imports the proved library, independently of these placeholders. Palomar's

comparator checks that the proved statements are exactly those of `Challenge.lean` and replays the proofs through Lean's kernel and independent kernels. The thirteen compared theorems are marked † in Table 1. The registry entry [7] records this verification for commit `40279dd`.

Table 1: Main results and Lean declarations, all in the namespace `ShortGCD`. Names marked † are compared against `Challenge.lean`.

| Paper | Lean declaration(s) |
|---|---|
| Theorem 1 | `uniform_real_bound`, `sizeCost_isBigO` |
| Normalisation (1) | `G_normalize` |
| Lemma 2 | `small_remainder` |
| Lemma 3 | `prefix_pairs_distinct`, `residue_card_sq` |
| Corollary 4 | `uniform_real_bound_six`†, `uniform_intrinsic_bound_six`† |
| Proposition 7 | `callsRem_eq_D`, `G_eq_D`, `callsCoprimeness_eq_callsRem`, `gcd_certificate_lower`, `callsGcd_le`, `callsRemGcd_le_callsGcd`, `exists_certifiesGcd`†, `G_le_callsRemGcd`†, `G_le_callsGcd`†, `callsCoprimeness_eq_G`† |
| Remark 8 | `callsCoprimeness_le`, `exists_certifiesCoprimeness`† |
| Corollary 9 | Cited Theorem 4F.1; not formalised here |
| Corollary 10 | `not_conjecture_3_4`†, `not_main_conjecture_4F5`†, `not_main_conjecture_4F5_Neps`†; literal versions `not_conjecture_3_4_literal`†, `not_main_conjecture_4F5_literal`† |
| Proposition 11 | `AlmostAll.card_D_le`, `AlmostAll.tendsto_card_within_loglog`, `tendsto_card_callsCoprimeness_loglog`† |
| Theorem 12: cost | `RegisterProgram.maxCost_step`, `RegisterProgram.Fixed.limsup_profile_le` |
| Theorem 12: syntax | `RegisterProgram.Syntax.eval_head_correct`, `RegisterProgram.Syntax.eval_head_exists`, `RegisterProgram.Syntax.arbitrarily_small_program` |
| Operational correspondence | `RegisterProgram.Syntax.Machine.eval_iff_steps`, `RegisterProgram.Syntax.Machine.arbitrarily_small_program` |
| Theorem 13 | `TailProgram.maxCost_step`, `TailProgram.Fixed.limsup_profile_le`, `TailSource.executes_iff`, `TailSource.executes_correct`, `TailSource.arbitrarily_small_coefficient` |

**Arithmetic bounds.** Costs are lengths of explicit remainder derivations. The finite theorem `sizeCost_step` proves the recurrence from natural-number size parameters; it does not assume a recurrence for the cost function. For the initial Big-O proof, Lean uses $j = \lfloor \log n \rfloor$, $p = \lfloor j/4 \rfloor$ and $h = 2^{2p} + 1$ in place of $h = \lceil \sqrt{n} \rceil$, and checks the integer rounding estimates before passing to real logarithms. The refinement uses the tunable parameters of Corollary 4 and checks the rounding limits preserving coefficient 6. For Proposition 11, Lean takes $B = a^{2^{-d}}/2$ separately for each pair instead of splitting into dyadic ranges. It proves the stated constant 40 and the exceptional-set limit for every fixed $\varepsilon > 0$, with threshold $(1 - \varepsilon) \log \log N$.

**Programs and machine computations.** Generator runs translate into evaluations of finite first-order syntax with exactly the same remainder count, including final verification. Every terminating evaluation is correct, and a successful evaluation exists on every input. Lean also proves exact-cost equivalence with a specialisation of the book's two-stack machine: arguments are evaluated right to left, zero tests and branches are separate transitions, and only remainder transitions

are charged [6, Table 1, pp. 75–76]. The forward proof embeds each argument evaluation in its calling context; the reverse proof decomposes a convergent computation into its argument and body evaluations. Closures represent substitution in machine states, not extra program data. This checks the language and program family used here; the repository README describes the Python compiler and interpreter and their relation to the Lean syntax.

The `RegisterProgram` declarations concern the generator of Theorem 12, whose nested calls use a stack. For Theorem 13, Lean defines the register program itself (`TailSource.Step`): finitely many lines, work registers, two input registers that no command writes, and no stack. Each command is an assignment $x_l := \mathrm{rem}(x_i, x_j)$ or a branch or halt on a single register, and each remainder evaluation is charged once, including the two verification calls. Every convergent execution returns the gcd, and one exists on every input. The cost proofs use a generator that may also relabel all registers at once; `TailSource.run_trace_relabel` replays such runs by single-register commands at the same cost, so no copying is needed.

## Declaration of AI-generated research and authorship

This manuscript is the product of an AI-driven research process. During a broader investigation initiated by the human author, an AI system identified the principal result. The author then instructed it to formalise the result in Lean and to prepare this manuscript. The AI systems developed the result and its presentation, including the mathematical exposition, the formalisation, and the organisation and writing of the text; the human author wrote no part of the manuscript. The research, the Lean development and the manuscript were produced with GPT-6-Astra (OpenAI), used through Codex. Claude Opus 5.5 (Anthropic), used through Claude Code, carried out review and editing passes and packaged the formalisation for the Palomar registry, which included writing `Challenge.lean`, the lower-bound module and a proof change needed by the independent kernels. GPT-6-Astra, through Codex, later revised the exposition and expanded the computational checks.

The Lean development gives a machine-checkable version of the main claims, subject to its trusted foundations (Lean's kernel, mathlib, and the axioms listed in Appendix A) and to the correspondence between its definitions and the book's notions (Lemma 6).

This declaration distinguishes the provenance of the work from conventional AI-assisted writing. The human author accepts responsibility for the submission and for the accuracy of this disclosure.

## References


[1] Nader H. Bshouty. *Euclidean GCD algorithm is not optimal.* Research Report 1989-376-38, Department of Computer Science, University of Calgary, December 1989. doi:10.11575/PRISM/30475.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

[3] Donald E. Knuth. *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, Reading, MA, 1997.

[4] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pp. 367–381. ACM, 2020. doi:10.1145/3372885.3373824.

[5] Yiannis N. Moschovakis. *Absolute lower bounds for uniform processes*. Unpublished lecture notes (first draft), January 25, 2012. https://www.math.ucla.edu/~ynm/nadalb.pdf.

[6] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic, vol. 48. Cambridge University Press and Association for Symbolic Logic, 2019. doi:10.1017/9781108234238.

[7] Palomar registry. Entry PALOMAR-2026-10-23-000012, version 1: *The Euclidean algorithm is not optimal among uniform processes*. Registered October 23, 2026, for commit `40279dd42196cc2f9dd6c14ecc3844b6822bff44` of the repository. https://palomar-registry.org/entry?id=PALOMAR-2026-10-23-000012&version=1.

[8] J. Stein. Computational problems associated with Racah algebra. *Journal of Computational Physics* 1(3):397–405, 1967. doi:10.1016/0021-9991(67)90047-2.

[9] Lou van den Dries and Yiannis N. Moschovakis. Is the Euclidean algorithm optimal among its peers? *Bulletin of Symbolic Logic* 10(3):390–418, 2004. doi:10.2178/bsl/1102022663.

[10] Lou van den Dries and Yiannis N. Moschovakis. Arithmetic complexity. *ACM Transactions on Computational Logic* 10(1), Article 2, 49 pp., 2009. doi:10.1145/1459010.1459012. Preprint: https://www.math.ucla.edu/~ynm/papers/ac.pdf.